

# Configuration Reference

**Owner:** Priya (Engineering)

**Status:** Draft with Martha’s user advocacy input for team review

**Date:** July 2026

**Source:** Expands the partial parameter list in the DocAlign 1.0 engineering handoff

## Purpose

This document specifies the complete set of parameters for `docalign.config.yaml`, the file that governs DocAlign's behavior for a repository. It expands the partial list given in the engineering handoff with types, defaults, constraints, and behavior notes.

The file lives at the repository root. It is read at the start of every analysis.

---

## Parameter summary

| Parameter                        | Type             | Default                           | Required |
|----------------------------------|------------------|-----------------------------------|----------|
| <code>integration</code>         | string (enum)    | —                                 | Yes      |
| <code>source_paths</code>        | array of strings | all paths                         | No       |
| <code>doc_paths</code>           | array of strings | all paths                         | No       |
| <code>excluded_paths</code>      | array of strings | empty                             | No       |
| <code>auto_remediate</code>      | boolean          | <code>true</code>                 | No       |
| <code>severity_thresholds</code> | object           | <code>{blocking: critical}</code> | No       |
| <code>notification_rules</code>  | array of objects | empty                             | No       |

---

## Parameters

---

## integration

**Type:** string (enum: `github`, `jenkins`) · **Required:** No default

Declares which CI/CD system DocAlign is running in. This determines how DocAlign authenticates, posts findings, and opens documentation pull requests.

This parameter is required and has no default. DocAlign cannot infer the correct value, and the two integrations use different mechanisms and credentials, so a wrong guess fails rather than degrades. If `integration` is absent, DocAlign exits with a configuration error.

```
1 integration: github
2
```

## source\_paths

**Type:** array of strings · **Default:** all paths in the repository

Directories and file patterns to analyze for source code. Accepts glob patterns.

If omitted, DocAlign analyzes the entire repository. This gives a working first run without configuration, at the cost of longer analysis and potentially more findings on a large repository. Narrow this parameter to reduce both.

Paths are relative to the repository root.

```
1 source_paths:
2   - "src/**/*.py"
3   - "src/**/*.java"
4
```

## doc\_paths

**Type:** array of strings · **Default:** all paths in the repository

Directories and file patterns where documentation lives. Accepts glob patterns.

Follows the same defaulting behavior as `source_paths`. DocAlign analyzes DITA, Markdown, and OpenAPI files found within these paths.

```
1 doc_paths:
2   - "docs/**/*.dita"
3   - "docs/**/*.md"
4   - "api/openapi.yaml"
5
```

## excluded\_paths

**Type:** array of strings · **Default:** empty (nothing excluded)

Directories and file patterns to skip. Accepts glob patterns.

**Exclusions take precedence.** A path matching both an include ( `source_paths` or `doc_paths` ) and `excluded_paths` is excluded. This follows the convention of `.gitignore` and similar tools, and is the safer default: exclusions are typically protective, so they should not be silently overridden.

```
1 excluded_paths:
2   - "vendor/**"
3   - "**/generated/**"
4   - "docs/legacy/**"
5
```

`auto_remediate`

**Type:** boolean · **Default:** `true`

Controls whether DocAlign generates suggested fixes for auto-remediable findings.

When `true` , accepting an auto-remediable finding produces a preview diff for confirmation.

When `false` , all findings are flagged for manual handling and no fixes are generated.

**Regardless of this setting, DocAlign never commits a change without explicit human confirmation of the preview diff.** The confirmation gate is structural, not configurable. This parameter controls whether fixes are *offered*, not whether they are *applied*.

```
1 auto_remediate: true
2
```

`severity_thresholds`

**Type:** object · **Default:** `{blocking: critical}`

Groups settings that key on severity level. In 1.0 it contains a single field.

| Field                 | Type                                                                            | Default               | Description                                 |
|-----------------------|---------------------------------------------------------------------------------|-----------------------|---------------------------------------------|
| <code>blocking</code> | string (enum: <code>critical</code> , <code>major</code> , <code>minor</code> ) | <code>critical</code> | The severity at which findings block merge. |

**Thresholds are inclusive of higher severities.** `blocking: major` means Major *and* Critical findings block merge — not Major alone. Severity is an ordered scale, and a threshold names the least severe level that triggers the behavior.

The default blocks only Critical findings, where documentation directly contradicts code behavior and a user following the documentation would get an error or wrong result.

```
1 severity_thresholds:
```

```
2 blocking: critical
3
```

## notification\_rules

**Type:** array of objects · **Default:** empty

Routes findings to recipients. Each rule pairs a condition with a recipient.

| Field    | Type                                  | Required | Description                                                 |
|----------|---------------------------------------|----------|-------------------------------------------------------------|
| paths    | array of strings                      | No       | Glob patterns.<br>The rule matches findings in these paths. |
| severity | string (enum: critical, major, minor) | No       | The rule matches findings at this severity or higher.       |
| notify   | string                                | Yes      | The recipient — a user, team, or channel handle.            |

A rule with no `paths` and no `severity` matches every finding. At least one of `paths` or `severity` should be present unless that is the intent.

**Rules are additive.** A finding matching multiple rules notifies all of them. Rule order is not significant, and adding a rule never suppresses an existing one. This makes routing predictable at scale: a new rule can broaden who is notified, but never narrows it.

The trade-off is that rules cannot express exclusions — there is no way to say "notify X except when Y." Narrow the conditions on each rule rather than attempting to subtract.

```
1 notification_rules:
2   # API docs team gets everything in their paths
3   - paths: ["docs/api/**"]
4     notify: "@api-docs-team"
5
6   # Platform writers get Major and Critical in their paths
7   - paths: ["docs/platform/**"]
8     severity: major
9     notify: "@platform-writers"
10
11  # Release managers see all Critical findings, regardless of path
12  - severity: critical
13    notify: "@release-managers"
14
```

## Monorepo example

Scoping by product path prevents one product's findings from reaching another product's team.

```
1 source_paths:
2   - "products/atlas/src/**"
3   - "products/beacon/src/**"
4
5 doc_paths:
6   - "products/atlas/docs/**"
7   - "products/beacon/docs/**"
8
9 notification_rules:
10  - paths: ["products/atlas/**"]
11    notify: "@atlas-docs"
12  - paths: ["products/beacon/**"]
13    notify: "@beacon-docs"
14
```

---

## Complete example

```
1 integration: github
2
3 source_paths:
4   - "src/**/*.py"
5   - "src/**/*.java"
6
7 doc_paths:
8   - "docs/**/*.dita"
9   - "docs/**/*.md"
10  - "api/openapi.yaml"
11
12 excluded_paths:
13   - "vendor/**"
14   - "**/generated/**"
15
16 auto_remediate: true
17
18 severity_thresholds:
19   blocking: critical
20
21 notification_rules:
22   - paths: ["docs/api/**"]
23     notify: "@api-docs-team"
24   - severity: critical
25     notify: "@release-managers"
26
```

---

## Open items

Two behaviors in this reference are unspecified and awaiting a product decision: the handling of findings that match no notification rule, and whether `integration` is required or detected.

Both are tracked in *DocAlign — Decisions & Open Questions* (Q3, Q4).